# Curvature-based Comparison of Two Neural Networks

Tao Yu
Department of Mathematics
Shanghai Jiao Tong University
Shanghai, China
Email: ydtydr@sjtu.edu.cn

Huan Long
Department of Computer Science and Engineering
Shanghai Jiao Tong University
Shanghai, China
Email: longhuan@cs.sjtu.edu.cn

John E. Hopcroft
Computer Science Department
Cornell Uniersity
New York, USA
Email: jeh@cs.cornell.edu

arXiv:1801.06801v1 [cs.LG] 21 Jan 2018

***Abstract*—In this paper we show the similarities and differences of two deep neural networks by comparing the manifolds composed of activation vectors in each fully connected layer of them. The main contribution of this paper includes 1) a new data generating algorithm which is crucial for determining the dimension of manifolds; 2) a systematic strategy to compare manifolds. Especially, we take Riemann curvature and sectional curvature as part of criterion, which can reflect the intrinsic geometric properties of manifolds. Some interesting results and phenomenon are given, which help in specifying the similarities and differences between the features extracted by two networks and demystifying the intrinsic mechanism of deep neural networks.**

## I. Introduction

In recent years, neural networks have been widely used in areas including pattern recognition and machine learning. While they deliver state-of-the-art accuracy on many artificial intelligence tasks, relatively less has been known about the intrinsic mechanism of their high performance. Simply using accuracy to score neural networks neglects some fundamental information hidden behind the complicated architectures. In this paper, we compare two deep neural networks with the intension to uncover how they acquire knowledge differently. To fulfill this goal, we propose the strategy of studying the manifold structure of given neural networks.

AlexNet [1] is the name of a convolutional neural network, which competed in the ImageNet Large Scale Visual Recognition Challenge in 2012. As mentioned in [1], AlexNet consists of five convolutional layers and three fully-connected layers with a final 1000-way softmax layer and has 60 million parameters and 650,000 neurons. We compare two neural networks AlexNet1 [2] and AlexNet2 [3] implemented in Caffe. These two neural networks have the same structure and close accuracy but different initial parameters and were trained independently. As a metaphor, they are twins who live in the same environment and have the same education, yet they could have different learning skills and acquire different knowledge in certain aspects. The work in this paper can be easily generalized to the comparison of any two neural networks with the same inputs. In contrast to existing approaches like [4], our framework focus on studying the intrinsic geometry of manifolds, namely, curvature information.

The remainder of this paper is organized as follows. Section II presents the geometric structure of neural network, and specifies the reason for taking curvature into account. Then a new SVD-based close data generating method is proposed, based on which we give the dimension estimation of manifolds in every fully connected layer in each AlexNet. Section III mainly shows the similarities and differences of two AlexNets by calculating Riemann curvature tensor and sectional curvature tensor. The traditional 2-norm distance based work is given for comparison. Section IV shows the related work which has been established in the last years. Finally, Section V gives our conclusions and directions for further research.

## II. Manifold and SVD-close Data

This section contains the mathematical conceptions necessary for our work, along with the strategy to generate a large amount of close data, which is the foundation for dimension estimation and local curvature calculation.

### A. Manifold and tangent space

A manifold $\mathcal{M}$ is a topological space that locally resembles Eulidean space near each point [10]. The concept of manifold is central to the modern manifold leaning theory, in which data are assumed to lie a manifold with a much lower dimension $d$ compared to that of the ambient space (the Euclidean space $\mathbb{R}^D$, $d \ll D$). The main goal is to determine $d$ and the embeding mapping $f : \mathbb{R}^d \to \mathcal{M}$ with certain local or global properties being preserved, i.e., dimension reduction. To identify $d$, we will need to determine the dimension of its tangent space. Let $\mathcal{M} \subset \mathbb{R}^D$ be a manifold. For every point $\mathbf{p} \in \mathcal{M}$, a tangent space is a real vector space that intuitively contains the possible directions in which one can tangentially pass through $\mathbf{p}$. The dimension of the tangent space at every point of a connected manifold is the same as that of the manifold itself.

In this paper, a high dimensional data point is an activation vector formed by the output of all neurons at some layers. For images that will be classified by AlexNet as the same label, we assume that corresponding activation vectors in a specific layer lie on the same manifold. The dimension and shape of the manifold are to be determined. For AlexNet, we will focus on three fully connected layers. As a notation,

activation vectors in the 6th (7th, 8th resp.) layer which will be recognized as label $\mathfrak{C}$ form a $\mathfrak{C}_6$-*manifold* ($\mathfrak{C}_7$,$\mathfrak{C}_8$-manifold resp.), or sometimes $\mathfrak{C}_i$ for short. Thus we have manifolds like $Persiancat_6$, $Goldfish_8$ etc.. It follows that $\mathfrak{C}_6, \mathfrak{C}_7 \subseteq \mathbb{R}^{4096}$, while $\mathfrak{C}_8 \subseteq \mathbb{R}^{1000}$. For each $\mathfrak{C}_i$-*manifold*, we will need to find the dimension of the tangent space at each point.

However, with the intrinsic curse of the high dimension [5], the activation vectors of two randomly selected images will almost certainly be far from each other. As to the study of manifold, the corresponding challenge is to find many close data points around any point $\mathbf{p} \in \mathfrak{C}_i$. They will form a local patch which can approximate the tangent space $T_{\mathbf{p}}\mathcal{M}$. Traditional augmentation consists of using a combination of affine transformations to generate close images, such as cropping, rotating, flipping input images and stochastic methods [6], [7]. However, previous augmentation methods aim at increasing the accuracy and reducing overfitting. When we try to approximate the tangent space on manifolds, one of the drawbacks of the previous methods is that they could produce data points that are not close enough to the given point $\mathbf{p}$. Instead in Section II-C we propose a new close data generating strategy which succeeds in producing a large amount of close data points on manifold efficiently.

### B. Riemannian manifold and curvature

A manifold will become a metric space when it is equipped with a metric. A Riemannian manifold $\langle \mathcal{M}, g \rangle$ is a manifold $\mathcal{M}$ endowed with an inner product $g_{\mathbf{p}}$ at the tangent space $T_{\mathbf{p}}\mathcal{M}$ at each point $\mathbf{p}$ that varies smoothly from point to point in the sense that if $X$ and $Y$ are differentiable vector fields on $\mathcal{M}$, then $\mathbf{p} \mapsto g_{\mathbf{p}}\left(X(\mathbf{p}), Y(\mathbf{p})\right)$ is a smooth function [10]. Three concepts closely related to the manifold-based comparison of neural network are the followings:

*Definition 1 (Riemann Curvature [8]):* Let $\langle \mathcal{M}, g \rangle$ be a Riemannian manifold and $\nabla$ the Riemannian connection. The curvature tensor is a $(1,3)-$tensor defined by

$$\mathcal{R}(X,Y)Z = \nabla_X \nabla_Y Z - \nabla_Y \nabla_X Z - \nabla_{[X,Y]} Z,$$

on vector fields $X, Y, Z$.

Using Riemannian metric $g$, $\mathcal{R}(X,Y)Z$ can be changed to a $(0,4)-$tensor [8]:

$$\mathcal{R}(X,Y,Z,W) = g\left(\mathcal{R}(X,Y)Z, W\right).$$

Note that this value is linear with respect to $X, Y, Z, W$, so we only need to determine this value in terms of a basis of $T_{\mathbf{p}}\mathcal{M}$.

*Definition 2 (Sectional curvature [8]):* Let $\langle \mathcal{M}, g \rangle$ be a Riemannian manifold, $p \in \mathcal{M}$, $u, v \in T_p\mathcal{M}$ are two linearly independent tangent vectors, the sectional curvature of the plane $\mathbb{R}u + \mathbb{R}v$ will be defined as

$$K(u,v) = \frac{R(u,v,u,v)}{\langle u,u \rangle \langle v,v \rangle - \langle u,v \rangle^2}.$$

where $R$ is the Riemann curvature tensor.

*Definition 3 (Local Isometry [9]):* Let $\langle \mathcal{M}, g \rangle$ and $\langle \mathcal{N}, h \rangle$ be two Riemannian manifolds where $g$ and $h$ are Riemannian metrics on them. For a map between manifolds $F : \mathcal{M} \to \mathcal{N}$, $F$ is called local isometry if $h(dF_{\mathbf{p}}(v), dF_{\mathbf{p}}(v)) = g(v,v)$ for all $\mathbf{p} \in \mathcal{M}$, $v \in T_{\mathbf{p}}\mathcal{M}$. Here $dF$ is the differential of $F$.

Intrinsic geometry will be preserved under local isometric mapping, like Riemann curvature tensor and sectional curvature tensor. This is one reason for us to include curvature tensor. Another advantage of curvatures is that they can reflect intrinsic properties of given manifold, such as the extent to which the metric tensor is not locally isometric to that of Euclidean space, which shows the deviation of the manifold from being a flat Euclidean-space. In addition, sectional curvatures can be used to determine geometric properties of the manifold such as convexity. As for AlexNet, though the manifold of the output layer can be proved to be convex, the $\mathfrak{C}_i$ $(i = 6, 7, 8)$ manifolds could be highly curved according to the complication of the network. In this case the 2-norm distance based methods become less reasonable.

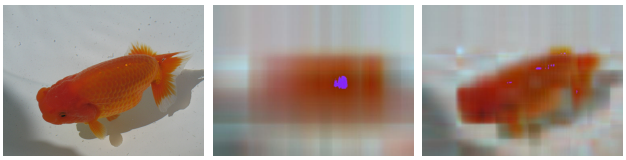
In brief, to have a more convincing comparison between two neural networks, we propose the strategy of comparing Riemann/sectional curvature tensor of the manifolds on the same layer of two different networks with the same input. Meanwhile the traditional Euclidean-space based method and experiments are implemented for comparative analysis. Details will be given in Section II-B.

### C. Generating close data

As we have mentioned earlier, activation vectors corresponding to random images are far from each other with high probability. Actually, according to our experiment, for the original images in ImageNet, the mean Euclidean distance between activation vectors on AlexNet1 (similar for AlexNet2) are around 320, 100 and 90 for $\mathfrak{C}_i$ $(i = 6, 7, 8)$ respectively. In order to approximate the tangent space, we need a large amount of close data around any given point $\mathbf{v}$. Our trick is to treat each image as a matrix (actually three matrix corresponding to the R,G,B channels resp.), then use matrix decomposition to generate enough close data, or formally:

*Definition 4 (Original/Derivative image):* Given a real matrix $I$ of size $m \times n$, then $I = U\Sigma V^T$ by singular value decomposition, where $U, V$ are real orthonormal matrices of size $m \times m$ and $n \times n$ respectively, and $\Sigma$ is a $m \times n$ rectangular diagonal matrix with non-negative real singular values $\{\lambda_1, \lambda_2, \cdots, \lambda_n\}$ on the diagonal in descending order. We call the image corresponding to $I$ the original image. The image corresponding to matrix $I'$ is called derivative image of $I$ if $I' = U\Sigma' V^T$, where $\Sigma'$ is a rectangular diagonal matrix with diagonal elements being $\{\alpha_1, \alpha_2, \cdots, \alpha_n\}$, where $\alpha_i \in \{0, \lambda_i\}$ for $i \in \{1, 2, \cdots n\}$.

Intuitively speaking, the derivative images derives from setting some of the singular values of the original matrix to zero. As the essence of SVD is discriminating noise, we are getting rid of a small part of noise from the original image. In our experiment, $m, n$ are around 500. To keep the closeness, we select the last $k_1, k_2, k_3$ $(1 \leq k_1, k_2, k_3 \leq 22)$ singular values of all for each R,G,B matrix of the primitive image to be zero. For each image we generate 10,648 close images.



(a) (b) (c)

Fig. 1: Images with different singular values: (a) Original $Goldfish$ image with all singular values included. (b) Derivative image of (a) with the first 3 singular values being kept. (c) Derivative image of (a) with the first 7 singular values being kept. It turns out that Alexnet classify (a) as a $Goldfish$ with probability 0.999988, (b) as a $Goldfish$ with probability 0,218679 and (c) as a $Goldfish$ with probability 0.900654. On the contrary, it is hard for human to say that (b) and (c) are of class $Goldfish$ for sure.

These images are inputs to both AlexNets to get the manifold. For given $\mathfrak{C}_i$ $(i = 6, 7, 8)$, the distance in a local patch is less than 0.9, 0.5 and 0.3 respectively, much smaller than that of the original data.

An interesting observation in our experiment is that we find that AlexNet seems to be more sensitive to singular values than human. As an example, for the *Goldfish* image in Fig.1(a)[1], Alexnet only needs the first $3 \sim 7$ largest singular values to recognize it correctly with high probability, which is almost impossible to human. As a matter of fact, for the class of *Goldfish* images, we find 288 out of the total 1236 images on which AlexNet remarkably outperforms human.

### D. Dimension estimation

For every original image and its activation vector in the $i$-th fully connected layer $(i = 6, 7, 8)$ , we create many close images by using above strategy, and run AlexNet to get their activation vectors in $\mathfrak{C}_i$. As all the vectors are close enough to form a local patch, we can now use this local patch to approximate the tangent space at given point. More precisely, to AlexNet1, for an original image $J \in \mathfrak{C}$, it will be mapped to a point $\mathbf{v} \in \mathbb{R}^{4096}$ on the $\mathfrak{C}_6$-manifold. Based on $J$ we get $n$ $(n = 10,648)$ derivative images $J_1, \cdots J_n$, whose activation vectors in $\mathfrak{C}_6$-manifold will be $\mathbf{v}_1^1, \ldots, \mathbf{v}_n^1 (\mathbf{v}_n = \mathbf{v})$ respctively. Then we have the following data matrix which represents the local patch around $\mathbf{v}$:

$$X_J^1 = \left[\mathbf{v}_1^1, \mathbf{v}_2^1, \ldots, \mathbf{v}_n^1\right]. \tag{1}$$

It is a matrix of size $4096 \times n$. We then use PCA to estimate the dimension of the $T_{\mathbf{v}}\mathfrak{C}_6$ (also the dimension of the $\mathfrak{C}_6$-manifold). Similarly, we can get $X_J^2$ and apply the same steps to AlexNet2. We find that if we keep the same percent of eigenvalues of the covariance matrix, the approximate dimension of the two AlexNets will be the same. Then we repeat the work on every original images in class $\mathfrak{C}$. It turns out that to the images of the same class, the approximate dimensions are nearly the same.

As an example, for $Goldfish$ label, in table I we show the mean and max value of the approximate dimension, where we take the first 90% of the sum of all eigenvalues of the covariance matrix.

| Manifold | $Mean_{A1}$ | $Mean_{A2}$ | $Max_{A1}$ | $Max_{A2}$ |
|---|---|---|---|---|
| $Goldfish_6$ | 11.78 | 11.63 | 14 | 14 |
| $Goldfish_7$ | 10.20 | 10.02 | 13 | 13 |
| $Goldfish_8$ | 7.83 | 7.67 | 10 | 10 |

TABLE I: By keeping the first 90% of the eigenvalues of the covariance matrix, the mean and max values of the approximate dimension for 150 local patches of two AlexNets.

Note that the dimension of each layer is nearly the same for two Alexnets, also it decreases as it gets nearer to the output layer. This phenomenon dovetails nicely to the intuition that after the work of each layer, the neural network uses fewer essential features to do classification and becomes more and more certain about the attribution of the input.

## III. Comparing Two AlexNets

For the two local patches in two Alexnets generated by the same image, we adopt different methods to compare them.

### A. Euclidean distance based method

Given $J$, $X_J^1$ and $X_J^2$ have been defined in (1) (for two AlexNets). Next, we use PCA to get the dimension-reduced data matrix named as $\mathcal{DR}(X_J^1)$ $(\mathcal{DR}(X_J^2))$ from the original patch, which is of size $d \times n$ for each layer, where $d$ is the max value estimated by PCA in the table I.

We first calculate some specific data of these two local patches on original data. For example, the mean length of each activation vector $||\mathbf{v}_i||_2$, the mean distance of activation vectors in the same patch $d_{ij}$ and the mean distance of activation vectors corresponding to the same image in two patches $d(\mathbf{v}_i^1, \mathbf{v}_i^2)$, which are listed in TABLE II.

It's obvious that the mean lengths of activation vectors corresponding to the same image are similar, also the tiny difference in distance matrix for two patches shows the closeness of the manifolds embeded in two AlexNets, which brings a question: Could it be the case that there exists a linear transformation between two AlexNets? It would be surprising if the answer is positive for the two neural networks were trained independently. To figure out the answer to this question, we proceed the work in two directions by trying to find a transformation $\mathfrak{T}$ composed of a scaling and rotation matrix $T$ and a translation vector $b$ which can minimize the root-mean-square error (RMSE) for both zero-mean normalized original data $\mathcal{N}(X_J)$ and dimension reduced data $\mathcal{N}(\mathcal{RD}(X_J))$, denoted as $L_{O-RMSE}$ and $L_{RD-RMSE}$:

1) $\mathcal{N}(X_J^1)$ to $\mathcal{N}(X_J^2)$;
2) $\mathcal{N}(\mathcal{RD}(X_J^1))$ to $\mathcal{N}(\mathcal{RD}(X_J^2))$.

where the root-mean-square error (RMSE) is given as follows:

$$L = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(T\mathcal{N}(\mathbf{v}_i^1) + b - \mathcal{N}(\mathbf{v}_i^2))^T(T\mathcal{N}(\mathbf{v}_i^1) + b - \mathcal{N}(\mathbf{v}_i^2))}$$

[1]All images in this paper can be found in the folder n01443537 corresponding to $Goldfish$ in ImageNet.